

Express Js Notes

1. Introduction to Express.js

What is Express.js?

Express.js is a lightweight, fast, and flexible web application framework built on top of **Node.js**.

It helps developers create:

- Web Applications
- RESTful APIs
- Backend Servers
- Full Stack Applications

Express simplifies server-side development by providing powerful features like routing, middleware, request handling, and response management.

Definition

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building web and mobile applications.

Why Do We Need Express.js?

Creating servers using only Node.js requires writing more code.

Using Node.js HTTP Module

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000);
```

Problems

- More boilerplate code
- Difficult routing
- Manual request handling

- Hard to manage large applications
-

Using Express.js

```
const express = require("express");
```

```
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Hello World");  
});
```

```
app.listen(3000);
```

Benefits

- Less code
 - Easy routing
 - Middleware support
 - Better project structure
 - Faster development
-

Features of Express.js

1. Routing

Routes determine how the application responds to client requests.

```
app.get("/", (req, res) => {  
  res.send("Home Page");  
});
```

2. Middleware Support

Middleware functions execute between request and response.

```
app.use((req, res, next) => {  
  console.log("Middleware Executed");  
  next();  
});
```

3. REST API Development

Express is widely used for creating APIs.

```
app.get("/users", (req, res) => {  
  res.json({ message: "User List" });  
});
```

4. Database Integration

Can easily connect with:

- MongoDB
 - MySQL
 - PostgreSQL
 - Oracle
 - SQLite
-

5. Template Engine Support

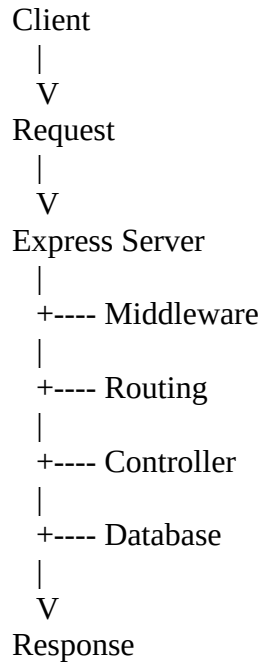
Supports:

- EJS
 - Pug
 - Handlebars
-

Advantages of Express.js

Advantage	Description
Fast Development	Less coding effort
Lightweight	Minimal framework
Easy Routing	Simple route creation
Middleware Support	Request processing
REST API Development	Excellent API framework
Large Community	Huge support available
Database Integration	Works with many databases
Scalable	Suitable for large applications

Express.js Architecture



2. Installing Express.js

Step 1: Create Project Folder

```
mkdir ExpressApp
```

Step 2: Move Into Folder

```
cd ExpressApp
```

Step 3: Initialize Node Project

```
npm init -y
```

This creates:

```
package.json
```

Step 4: Install Express

npm install express

What Happens After Installation?

NPM downloads Express and stores it inside:

node_modules

Verify Installation

npm list express

Output:

express@5.x.x

Project Structure

ExpressApp

- node_modules
- package.json
- package-lock.json
- server.js

Purpose of Each File

File	Purpose
node_modules	Stores installed packages
package.json	Project information
package-lock.json	Dependency versions
server.js	Main server file

3. Creating First Express Server

Step 1: Create server.js

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.send("Welcome to Express");
});

app.listen(3000, () => {
  console.log("Server Running");
});
```

Step 2: Run Server

node server.js

Output:

Server Running

Step 3: Open Browser

http://localhost:3000

Output:

Welcome to Express

Understanding the Code

Import Express

```
const express = require("express");
```

Loads Express package.

Create Application

```
const app = express();
```

Creates an Express application object.

Create Route

```
app.get("/", (req, res) => {  
  res.send("Welcome to Express");  
});
```

Handles GET requests.

Start Server

```
app.listen(3000);
```

Runs server on port 3000.

Important Terms

Component	Meaning
express()	Creates Express application
app	Express application object
app.get()	Handles GET requests
req	Request Object
res	Response Object
listen()	Starts server
port	Communication endpoint

Server Flow

```
Browser  
|  
V  
GET /  
|  
V  
Express Route
```

|
V
Response Sent

4. Request and Response Objects

Whenever a client sends a request, Express automatically creates:

req
res

Request Object (req)

The request object contains all information about the client's request.

```
app.get("/", (req, res) => {  
  console.log(req.method);  
  console.log(req.url);  
  
  res.send("Done");  
});
```

Common Request Properties

req.method

Returns HTTP method.

```
console.log(req.method);
```

Output:

GET

req.url

Returns URL path.

```
console.log(req.url);
```

Output:

/

req.headers

Returns request headers.

```
console.log(req.headers);
```

Output Example:

```
{  
  host: "localhost:3000",  
  connection: "keep-alive"  
}
```

req.params

Gets route parameters.

```
app.get("/user/:id", (req, res) => {  
  res.send(req.params.id);  
});
```

URL:

/user/101

Output:

101

req.query

Gets query parameters.

```
app.get("/search", (req, res) => {  
  res.send(req.query.name);  
});
```

URL:

/search?name=Prathamesh

Output:

Prathamesh

req.body

Used to access request data sent by client.

req.body

Example:

```
{  
  "name": "Prathamesh"  
}
```

Response Object (res)

The response object is used to send data back to the client.

1. res.send()

Sends text, HTML, or objects.

```
res.send("Hello");
```

Output:

Hello

2. res.json()

Sends JSON response.

```
res.json({  
  name: "Prathamesh",  
  course: "AIDS"  
});
```

Output:

Prathamesh Arvind Jadhav

```
{  
  "name": "Prathamesh",  
  "course": "AIDS"  
}
```

3. **res.status()**

Sets HTTP status code.

```
res.status(404).send("Not Found");
```

Output:

Not Found

Status:

404

4. **res.sendFile()**

Sends file to browser.

```
res.sendFile(__dirname + "/index.html");
```

5. **res.redirect()**

Redirects to another route.

```
res.redirect("/home");
```

5. **Routing in Express.js**

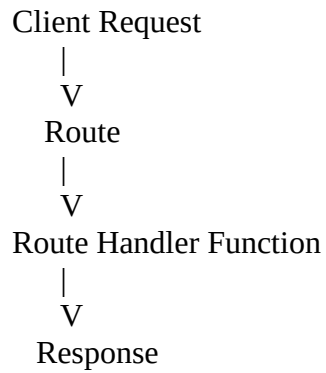
What is Routing?

Routing refers to determining how an application responds to a client's request for a specific URL (endpoint) and HTTP method.

In simple words:

Routing decides which function should execute when a user visits a particular URL.

Routing Flow



Syntax of Route

`app.METHOD(PATH, CALLBACK_FUNCTION);`

Example

```
app.get("/", (req, res) => {  
  res.send("Welcome");  
});
```

Where:

Component	Meaning
app	Express Application
get()	HTTP Method
"/"	URL Path
callback	Function executed

HTTP Methods in Express

Method	Purpose
GET	Fetch Data
POST	Insert Data
PUT	Update Data
DELETE	Delete Data

GET Route

Used to retrieve data from server.

```
app.get("/home", (req, res) => {  
  res.send("Home Page");  
});
```

URL

http://localhost:3000/home

Output

Home Page

POST Route

Used to send data to server.

```
app.post("/add", (req, res) => {  
  res.send("Data Added");  
});
```

Output

Data Added

PUT Route

Used to update existing data.

```
app.put("/update", (req, res) => {  
  res.send("Updated");  
});
```

Output

Updated

DELETE Route

Used to remove data.

```
app.delete("/delete", (req, res) => {  
  res.send("Deleted");  
});
```

Output

Deleted

Multiple Routes Example

```
const express = require("express");  
  
const app = express();  
  
app.get("/", (req, res) => {  
  res.send("Home Page");  
});  
  
app.get("/about", (req, res) => {  
  res.send("About Page");  
});  
  
app.get("/contact", (req, res) => {  
  res.send("Contact Page");  
});  
  
app.listen(3000);
```

Route Matching

Express checks routes from top to bottom.

```
app.get("/home", (req, res) => {  
  res.send("Home");  
});
```

Request:

/home

Express finds matching route and executes it.

Route Methods Summary

Method	Description
app.get()	Fetch Data
app.post()	Add Data
app.put()	Update Data
app.delete()	Remove Data
app.all()	Handles All Methods

app.all()

Executes for all HTTP methods.

```
app.all("/test", (req, res) => {  
  res.send("Works for all methods");  
});
```

6. Route Parameters

What are Route Parameters?

Route parameters allow us to create **dynamic URLs**.

Instead of:

```
/user/101  
/user/102  
/user/103
```

We can use one route.

Syntax

```
app.get("/user/:id", (req, res) => {  
});
```

:id is a dynamic parameter.

Example

```
app.get("/user/:id", (req, res) => {  
  const id = req.params.id;  
  res.send(`User ID: ${id}`);  
});
```

URL

/user/101

Output

User ID: 101

How req.params Works

```
console.log(req.params);
```

Output:

```
{  
  id: "101"  
}
```

Multiple Route Parameters

```
app.get("/user/:id/book/:bookId", (req, res) => {  
  res.send(  
    `User: ${req.params.id}  
    Book: ${req.params.bookId}`  
  );  
});
```

URL:

/user/101/book/501

Output:

User: 101
Book: 501

Route Parameter Flow

URL
|
V
/user/101
|
V
:id
|
V
req.params.id
|
V
101

7. Query Parameters

What are Query Parameters?

Query parameters are used to send additional information in the URL.

They appear after the ? symbol.

Structure

/search?name=Prathamesh

Where:

name = Key
Prathamesh = Value

Example

```
app.get("/search", (req, res) => {  
    const name = req.query.name;
```

Prathamesh Arvind Jadhav

```
    res.send(name);  
  });
```

URL:

/search?name=Prathamesh

Output:

Prathamesh

Accessing Multiple Query Parameters

URL:

/search?name=Prathamesh&course=AIDS

Code:

```
app.get("/search", (req, res) => {  
  res.send(  
    `Name: ${req.query.name},  
    Course: ${req.query.course}`  
  );  
});
```

Output:

Name: Prathamesh
Course: AIDS

req.query Object

```
console.log(req.query);
```

Output:

```
{  
  name: "Prathamesh",  
  course: "AIDS"  
}
```

Route Params vs Query Params

Route Params	Query Params
Part of URL path	After ? symbol
Required	Usually Optional
req.params	req.query
/user/101	/search?name=abc

Example Comparison

Route Parameter

/user/101
req.params.id

Query Parameter

/search?name=Prathamesh
req.query.name

8. Middleware

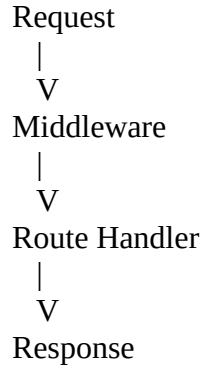
What is Middleware?

Middleware is a function that executes between the request and the response cycle.

It can:

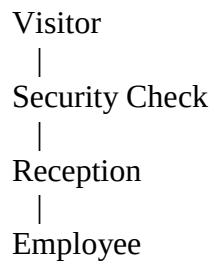
- Modify Request
 - Modify Response
 - Execute Code
 - Validate User
 - Log Requests
 - Handle Errors
 - End Request Cycle
-

Middleware Flow



Real Life Example

Imagine entering a company:



Security Check = Middleware

Middleware Syntax

```
function middleware(req, res, next) {  
  console.log("Middleware Executed");  
  next();  
}
```

Parameters

Parameter	Meaning
req	Request Object
res	Response Object
next()	Pass Control to Next Middleware

Using Middleware

```
app.use(middleware);
```

Example:

```
function logger(req, res, next) {  
    console.log("Request Received");  
    next();  
}  
  
app.use(logger);
```

Complete Example

```
const express = require("express");  
  
const app = express();  
  
function logger(req, res, next) {  
    console.log("Middleware Executed");  
    next();  
}  
  
app.use(logger);  
  
app.get("/", (req, res) => {  
    res.send("Home Page");  
});  
  
app.listen(3000);
```

Output:

```
Middleware Executed  
Home Page
```

Why next() is Important?

Without next():

```
function middleware(req, res, next) {  
  console.log("Middleware");  
}
```

Request gets stuck because Express never moves to the next function.

Correct Version

```
function middleware(req, res, next) {  
  console.log("Middleware");  
  next();  
}
```

Middleware Execution Order

```
app.use(middleware1);  
app.use(middleware2);  
app.use(middleware3);
```

Execution:

```
middleware1  
  ↓  
middleware2  
  ↓  
middleware3  
  ↓  
Route
```

9. Types of Middleware

1. Application Middleware

Prathamesh Arvind Jadhav

Runs for every request.

```
app.use((req, res, next) => {  
  console.log("Application Middleware");  
  next();  
});
```

Flow

```
Request  
↓  
Application Middleware  
↓  
Route  
↓  
Response
```

2. Route Middleware

Runs only for specific routes.

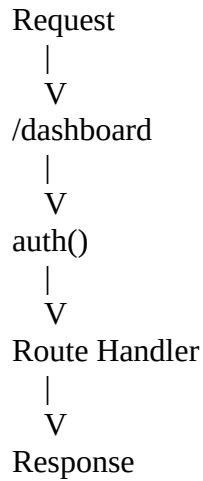
Example

```
function auth(req, res, next) {  
  console.log("Authentication");  
  next();  
}  
  
app.get("/dashboard", auth, (req, res) => {  
  res.send("Dashboard");  
});
```

Output:

```
Authentication  
Dashboard
```

Route Middleware Flow



3. Error Middleware

Handles application errors.

Syntax

```
app.use((err, req, res, next) => {  
    res.status(500).send("Error");  
});
```

Notice:

(err, req, res, next)

4 parameters are required.

Example

```
app.get("/", (req, res) => {  
    throw new Error("Something Wrong");  
});  
  
app.use((err, req, res, next) => {
```



```
res.status(500).send(err.message);  
});
```

Output:

Something Wrong

Types of Middleware Summary

Middleware	Purpose
Application Middleware	All Requests
Route Middleware	Specific Routes
Error Middleware	Error Handling
Built-in Middleware	Provided by Express
Third Party Middleware	Installed Packages

10. Built-in Middleware

Express provides built-in middleware functions.

express.json()

Purpose

Converts incoming JSON data into JavaScript objects.

Enable JSON Middleware

```
app.use(express.json());
```

Request Body

```
{  
  "name": "Prathamesh"  
}
```

Access Data

```
app.post("/user", (req, res) => {  
  console.log(req.body);  
  res.send(req.body.name);  
});
```

Output:

Prathamesh

Why `express.json()` is Needed?

Without it:

`req.body`

Output:

undefined

`express.urlencoded()`

Purpose

Parses data submitted from HTML forms.

Enable Middleware

```
app.use(express.urlencoded({ extended: true }));
```

HTML Form Example

```
<form method="POST">  
  <input type="text" name="username">  
  <button>Submit</button>
```

</form>

Access Form Data

```
app.post("/", (req, res) => {  
  console.log(req.body.username);  
  res.send(req.body.username);  
});
```

Difference Between JSON and URL Encoded

express.json()	express.urlencoded()
JSON Data	Form Data
API Requests	HTML Forms
application/json	application/x-www-form-urlencoded

11. Serving Static Files in Express.js

What are Static Files?

Static files are files that do not change dynamically, such as:

- HTML files
 - CSS files
 - Images
 - JavaScript files (frontend)
-

Example Folder Structure

```
public/  
├── index.html  
└── style.css
```

How to Serve Static Files

Code

```
app.use(express.static("public"));
```

How It Works

If you write:

`http://localhost:3000/index.html`

Express automatically looks inside the public folder.

Flow of Static Files

Browser Request



Express static middleware



public folder



`index.html / style.css`



Response

Example

public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello Express</h1>
</body>
</html>
```

public/style.css

```
h1 {  
  color: blue;  
}
```

12. Sending Files in Express.js

What is res.sendFile()?

It is used to send a specific file as a response.

Example Code

```
const path = require("path");  
  
app.get("/", (req, res) => {  
  res.sendFile(  
    path.join(__dirname, "index.html")  
  );  
});
```

Why use path.join()?

It ensures correct file path across all operating systems.

What is __dirname?

It gives the current directory path of the file.

Flow

```
Request "/"  
↓  
Express Server  
↓  
index.html file  
↓  
Sent to browser
```

Difference: Static Files vs sendFile()

Feature	express.static	res.sendFile
Purpose	Serve multiple files	Serve single file
Usage	Automatic routing	Manual control
Best for	CSS, JS, images	Specific HTML page

13. Express Router

What is Express Router?

Express Router is used to create **modular and organized routes**.

It helps split large applications into multiple files.

Why Router is Needed?

Without router:

- All routes in one file
- Hard to manage large projects

With router:

- Clean structure
 - Modular code
 - Easy maintenance
-

Step 1: Create users.js

```
const express = require("express");
```

```
const router = express.Router();
```

```
router.get("/", (req, res) => {  
  res.send("All Users");  
});
```

```
module.exports = router;
```

Step 2: Use Router in server.js

```
const express = require("express");  
const userRoutes = require("./users");  
const app = express();  
app.use("/users", userRoutes);  
app.listen(3000);
```

How Routing Works

Request:

/users

Flow:

```
app.use("/users")  
  ↓  
router.get("/")  
  ↓  
Response: "All Users"
```

Router Prefix Concept

```
app.use("/users", userRoutes);
```

URL	Result
/users	All Users
/users/profile	Profile route (if added)

Advantages of Router

- Clean Code
- Scalability
- Separation of Concerns

- Easy Debugging

14. REST API Concepts

What is REST?

REST stands for:

Representational State Transfer

It is an architecture used to design APIs.

REST API Features

- Stateless
 - Uses HTTP methods
 - Uses URLs as resources
 - Uses JSON format
-

CRUD Operations

Operation	HTTP Method
Create	POST
Read	GET
Update	PUT
Delete	DELETE

REST Concept Example

Users Resource:

/users

Action	Endpoint
Add User	POST /users
Get Users	GET /users
Update User	PUT /users/:id

Action	Endpoint
Delete User	DELETE /users/:id

15. REST API Example (Complete)

```
const express = require("express");
```

```
const app = express();
```

```
app.use(express.json());
```

```
let users = [];
```

CREATE (POST)

```
app.post("/users", (req, res) => {
```

```
    users.push(req.body);
```

```
    res.send("User Added");
```

```
});
```

READ (GET)

```
app.get("/users", (req, res) => {
```

```
    res.json(users);
```

```
});
```

UPDATE (PUT)

```
app.put("/users/:id", (req, res) => {
```

```
    res.send("Updated");
```

```
});
```

DELETE (DELETE)

```
app.delete("/users/:id", (req, res) => {
```

```
    res.send("Deleted");  
});
```

Full REST Flow

```
POST /users → Create  
GET /users → Read  
PUT /users → Update  
DELETE /users → Delete
```

16. HTTP Status Codes

What are Status Codes?

Status codes indicate the result of a request.

Common HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Server Error

Explanation

200 OK

Request successful.

```
res.status(200).json({  
  message: "Success"  
});
```

201 Created

Used after creating new resource.

```
res.status(201).send("User Created");
```

400 Bad Request

Client sent wrong data.

```
res.status(400).send("Invalid Request");
```

404 Not Found

Route not found.

```
res.status(404).send("Not Found");
```

500 Server Error

Internal server error.

```
res.status(500).send("Server Error");
```

Status Code Flow

Client Request



Server Processing



Status Code Generated



Response Sent

17. Error Handling in Express.js

What is Error Handling?

Error handling is used to manage runtime errors and prevent server crashes.

How Errors Work in Express

Express has a special flow for errors:

Route → next(err) → Error Middleware → Response

Example: Throwing an Error

```
app.get("/error", (req, res, next) => {  
  const err = new Error("Something Wrong");  
  next(err);  
});
```

What is next(err)?

- It skips normal routes
 - Directly goes to error-handling middleware
-

Error Handling Middleware

Syntax (Important □)

```
app.use((err, req, res, next) => {  
  res.status(500).json({  
    message: err.message  
  });  
});
```

Example Output

Request:

/error

Response:

```
{  
  "message": "Something Wrong"  
}
```

Error Flow Diagram

Request
↓
Route
↓
next(err)
↓
Error Middleware
↓
Response (500)

Key Points

- Error middleware must have 4 parameters
 - Always placed at the end of file
 - Used for centralized error handling
-

18. Environment Variables (.env)

What are Environment Variables?

They store sensitive and configurable data like:

- Port numbers
 - Database URLs
 - API keys
 - Secrets
-

Why Use .env?

- Security
 - Flexibility
 - Easy configuration changes
-

Step 1: Install dotenv

```
npm install dotenv
```

Step 2: Create .env File

```
PORT=5000
```

Step 3: Load .env in Server

```
require("dotenv").config();  
  
console.log(process.env.PORT);
```

Output

```
5000
```

Usage in Server

```
const express = require("express");  
  
require("dotenv").config();  
  
const app = express();  
  
const PORT = process.env.PORT || 3000;  
  
app.listen(PORT, () => {  
  console.log(`Server running on ${PORT}`);  
});
```

Key Points

Concept	Meaning
process.env	Access environment variables
.env file	Stores config values
dotenv	Loads variables into app

19. Third-Party Middleware (CORS)

What is CORS?

CORS = Cross-Origin Resource Sharing

It allows frontend apps to communicate with backend APIs.

Problem Without CORS

Frontend (React) → Backend (Node)

☐ Blocked by browser security

Solution: Enable CORS

Install CORS

```
npm install cors
```

Use CORS Middleware

```
const cors = require("cors");
```

```
app.use(cors());
```

What CORS Does

- Allows API requests from different domains
 - Enables frontend-backend communication
 - Prevents browser blocking
-

Example Flow

Frontend (React)



CORS Middleware



Express API



Response

20. Morgan Logger Middleware

What is Morgan?

Morgan is a logging middleware used to track HTTP requests.

Install Morgan

```
npm install morgan
```

Setup Morgan

```
const morgan = require("morgan");
```

```
app.use(morgan("dev"));
```

Output Example

GET / 200 5ms

POST /users 201 10ms

Morgan Formats

Format	Description
dev	Colored short logs
combined	Detailed logs
tiny	Minimal logs

Why Use Morgan?

- Debugging

- Monitoring requests
 - Development tracking
-

21. Authentication Basics (JWT)

What is Authentication?

Authentication verifies the identity of a user.

What is JWT?

JWT = JSON Web Token

It is used for secure authentication.

JWT Authentication Flow

Login
↓
Generate Token
↓
Send Token to Client
↓
Client Stores Token
↓
Access Protected Routes
↓
Server Verifies Token

Step-by-Step Explanation

1. Login User

User sends credentials.

2. Generate Token

Prathamesh Arvind Jadhav

Server creates JWT token.

3. Send Token

Token is sent to client.

4. Store Token

Client stores token in:

- localStorage
 - cookies
-

5. Protected Routes

User sends token with request.

6. Verify Token

Server checks validity.

Key Concept

Without valid token → Access denied

22. MVC Architecture

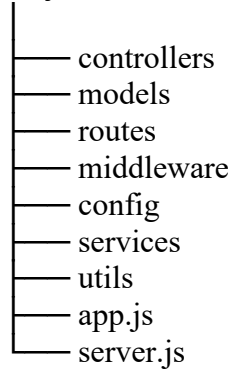
What is MVC?

MVC = Model View Controller

It is a design pattern used to structure applications.

MVC Folder Structure

Project



MVC Flow

Request



Route



Controller



Model



Database



Response

Explanation of Components

1. Model

- Represents database structure
- Handles data logic

2. View

- Frontend/UI (React, HTML, etc.)

3. Controller

- Business logic
 - Handles requests
-

MVC Flow Example

User Request → Controller → Model → Database → Response

Benefits of MVC

- Clean structure
 - Easy maintenance
 - Scalable codebase
 - Separation of concerns
-

23. Express Project Lifecycle

Complete Request Lifecycle

Client
↓
Request
↓
Route
↓
Middleware
↓
Controller
↓
Service
↓
Database
↓
Response

Step-by-Step Breakdown

1. Client

User sends request.

2. Request

HTTP request sent to server.

3. Route

Matches endpoint.

4. Middleware

Processes request (auth, logging, etc.)

5. Controller

Handles business logic.

6. Service Layer

Handles complex logic or reusable functions.

7. Database

Data stored or retrieved.

8. Response

Final output sent to client.

Real Example Flow

POST /users



auth middleware



controller (createUser)



service (validate data)



database (save user)



response (User Created)